GFX2D：基于 RISC-V 与 FPGA 异构架构的 2D 图形渲染加速引擎

——技术报告——

摘　要

二维图形渲染是嵌入式人机界面的基础负载。若完全由 CPU 逐像素搬运与混合，像素运算会占满处理器，帧率随画面元素数量骤降。本作品在 Ti60F225 FPGA 上实现了一套由 RISC-V 软件驱动的 2D 图形渲染加速系统：Sapphire 软核处理器运行自研弹幕射击游戏 MYRIAD，只负责游戏逻辑与生成 32 字节渲染命令表；自研 BitBlt 渲染引擎作为 AXI4 主机直接访问 DDR3，按命令完成图像块拷贝、纯色填充、硬件 Alpha 混合（12 个 DSP 混合核，每周期 4 像素）与颜色键控去背；扫描输出模块以双缓冲方式从帧缓冲取数，经 HDMI 以 640×480@60 Hz 显示。软件执行器与硬件流水线按同一份逐像素规格实现，板上同帧 CRC 对比证明两者逐像素等价。实测结果：普通游戏画面硬件后端稳定 60 FPS，同命令表的纯软件渲染约 2.5 FPS，加速约 24 倍；在开启 Alpha 混合与色键的条件下，自动搜索测得稳定 60 FPS 下同屏活动精灵数极限 N_max 为 16×16 弹型 1014 颗、8×8 弹型 1365 颗，3600 场（60 秒）确认零掉帧；整机占用 36009 个 XLR（59.2%）、105 块片上 RAM（41.0%）、16 个 DSP（10.0%），核心时钟时序余量为正。

第一部分　作品概述

1.1 功能与特性

本作品是一套完整的 2D 渲染加速系统，包含硬件渲染引擎、显示输出链路、板上软件平台和一个真实可玩的弹幕射击游戏。RISC-V 处理器运行游戏逻辑，将每帧的绘制需求编码为 32 字节渲染命令（块拷贝 BLIT、纯色填充 FILL、裁剪 SET_CLIP），写入 DDR3 命令表并写入 DOORBELL 寄存器触发引擎取指；FPGA 端的 2D 渲染引擎取指执行，以每周期 4 像素的吞吐完成像素搬运、12 通道 DSP Alpha 混合与色键去背，结果写入 DDR3 帧缓冲；扫描输出模块按场同步从帧缓冲取数，经双缓冲切换与 TMDS 编码后输出 HDMI。三个演示模式覆盖赛题全部要求：基础演示（大量方块随机移动、多张大图快速切换，屏幕同屏显示软/硬件两路帧率）、可玩游戏（默认开启 Alpha 与色键）、压力测试（自动搜索稳定 60 FPS 下的同屏精灵数量极限并显示结果）。

1.2 应用领域

本系统面向 CPU 逐像素渲染难以胜任、又无需 3D 管线的各类嵌入式图形场景。典型应用包括：高密度信息显示与仪表叠加（半透明面板、文字图层）、工业与医疗设备的图形化人机界面、游戏机与街机风格的娱乐终端、以及车载液晶仪表的图层合成。渲染引擎的命令接口带版本号与能力寄存器，混合方程以表格方式扩展，可在不改动流水线结构的前提下增加效果种类；配合上位机经串口转发输入，也可作为图形子系统集成进更大的 SoC 方案。赛题演示所用的弹幕游戏对渲染吞吐的要求高于多数仪表界面，因此本系统的性能余量可以平移到上述场景。

1.3 主要技术特点

第一，预算驱动的设计方法。开工前先用板上探针实测 DDR3 控制器的事务模型（突发上限 256 拍，读事务固定开销 27.4 周期、写 7.3 周期），得出“瓶颈在事务次数而非带宽”的结论，再据此定出帧缓冲行跨 4096 字节（行不跨 4 KB 边界）、4 像素并行、长突发优先的像素通路。第二，软硬同规格双实现。软件执行器与硬件流水线实现同一份逐像素规格（六种混合方程、÷255 舍入、裁剪与色键语义全部定稿），软件侧既是 PC 开发平台，也是硬件的金标准参照。第三，全链路自研闭环。从 PNG 素材打包、Flash 镜像合成、bootloader 引导、素材装载，到游戏逻辑、碰撞检测、命令生成、统计埋点，均为团队自行实现，整链在真实板卡上运行。第四，可观测性贯穿始终。硬件性能计数器（命令数、像素数、色键跳过数、读写突发与 beat 数、等待周期）与软件统计按同一统计定义设计，板上可直接对账。

1.4 主要性能指标

表 1　主要性能指标（实测）

指标	实测值
显示输出	640×480 @ 60 Hz，HDMI（TMDS），双缓冲无撕裂
渲染吞吐	每周期 4 像素，12 个 DSP 混合核 @ 100 MHz
帧率（普通游戏，硬件后端）	首秒预热后稳定 60 FPS
帧率（同负载，纯软件后端）	约 2.5 FPS，加速约 24 倍
同屏活动精灵极限 N_max	16×16 弹型 1014 颗；8×8 弹型 1365 颗（均经 3600 场确认零掉帧）
负载能力（过载档，约 2030 活动弹）	30.6 FPS，FIFO 零欠载
整机资源	36009 XLR（59.2%）、105 块片上 RAM（41.0%）、16 个 DSP（10.0%）
时序	core_clk setup 余量 +1.067 ns，hold +0.026 ns，无违例

1.5 主要创新点

- 以板上实测的 DDR3 事务模型反推架构：识别出固定事务开销（读约为写的 3.8 倍）才是并发瓶颈，用数据布局（行跨 4096 字节）与访问形态（长突发、读改写合并）直接消解，而非堆砌带宽。
- 同一份逐像素规格驱动软件与硬件两个实现，配以全定义域穷举校验（ALPHA 预乘路径 8,421,376 组逐值一致）与板上同帧 CRC 对比，使“加速但不改画”成为可证明的性质。
- N_max 指标的自动化测量：把赛题“稳定 60 FPS 下同屏活动元素极限”定义为可执行判据（倍增加盲到二分、候选档 3600 场确认），在真实游戏负载上自动搜索并记录，避免了人工演示式的定义争议。
- 命令 ABI 以表面表间接寻址并内建裁剪、越界、重叠检查与十步错误模型，错误码、错误定位与首错副作用语义完整，驱动无需信任输入。

1.6 设计流程

设计自底向上分四步推进：先以厂商工程为基线搭起可上板的硬件平台（下载、内存、显示、启动四条通路与回归判据）；再用探针程序实测 DDR3 地址规则、突发模型与多主机仲裁行为，形成预算；随后按预算实现渲染引擎（计算核心、AXI 事务层、几何层、控制层、顶层集成五个工作点，每个工作点单元仿真与板上判据通过后合入），软件侧同步在 PC 上以同一规格实现执行器与游戏；最后整机联调，以同帧 CRC、性能计数器对账与 N_max 自动搜索完成验收。每一步的判据先于实现写定，判据的有效性以故障注入测试（Fault Injection Testing）验证：对每条判据构造注入错误的对照版本，判据必须报出注入的故障方视为有效。

第二部分　系统组成及功能说明

2.1 整体介绍

系统由一颗软核处理器、三个自研硬件模块和标准存储/显示外设组成，全部装载于赛题指定的 Ti60F225 开发板。如图 1 所示，Sapphire SoC（VexRiscv RV32IM @ 100 MHz）运行游戏与驱动；自研的 GFX2D 渲染引擎与扫描输出模块 display 是 DDR3 的另外两个总线主机，三者经厂商 AXI 多路器 Axi_Mux（三进一出，128 位）共享 DDR3 控制器；axi_mux_guard 为处理器通路提供防饥饿保护。引擎与显示模块的控制寄存器、状态寄存器与性能计数器挂在 APB 总线（0xF810_0000 起），完成与错误经两条中断线上报处理器。

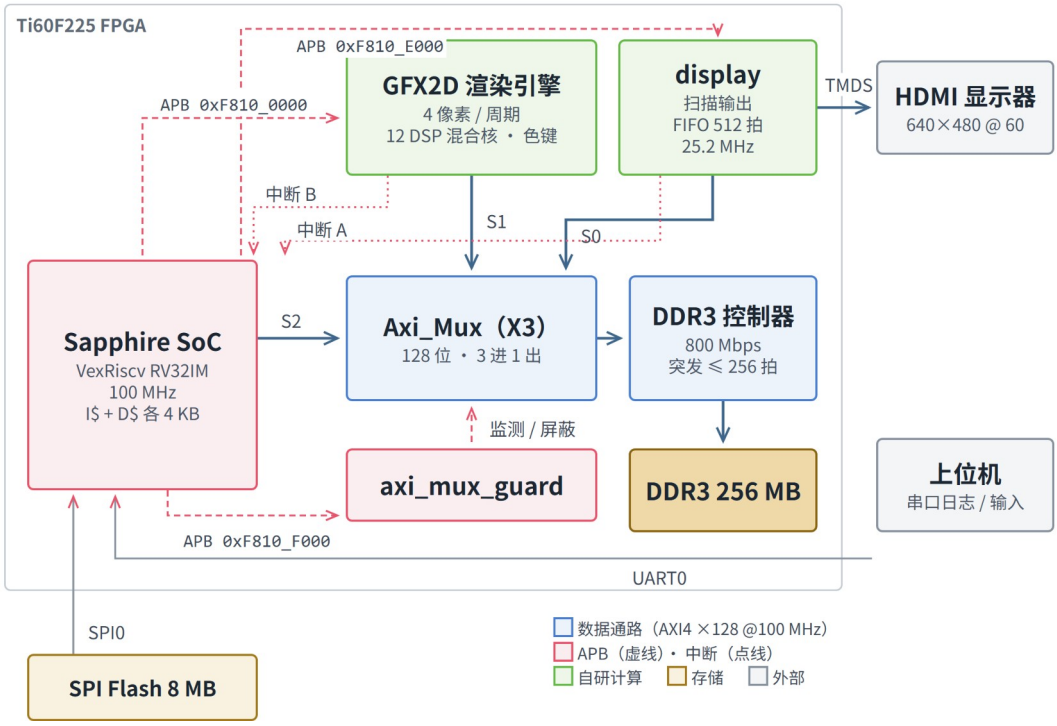


图 1　系统总体架构

三个总线主机的分工是刻意设计的：扫描输出是实时流，每个场必须取完 300 笔行读，故独占只读端口 S0 且每方向只保留一笔在途事务；渲染引擎是吞吐型负载，接 S1，读写各一笔在途，既保持流水满载，又避免处理器通路陷入

饥饿；处理器接 S2，由永久固化的 guard 模块监测地址等待超龄并暂时屏蔽引擎的新授权。这一分配在整机仿真中验证：引擎遵守一笔在途契约时，处理器地址等待不超过 414 个周期，画面扫描从不缺数。

一帧的完整数据流如下。处理器在帧 N 更新游戏状态（定点运算、碰撞检测、实体池压缩），生成约两千条 32 字节命令写入命令表，写 LIST_BASE 与 LIST_COUNT 后写入 DOORBELL 触发取指；引擎取指、检查、裁剪，把每个矩形展开成行段，经像素流水线读写 DDR3 完成混合；渲染完成触发中断 B，处理器请求换页，扫描输出在下一个场消隐期把前台指向新帧；HDMI 输出该帧。CPU 生成命令与引擎渲染上一帧命令同时进行，生成与渲染两段流水并行。

2.2 硬件系统介绍

2.2.1 硬件整体与存储地图

Ti60F225 具备 60.8K 逻辑单元、256 块 10 Kbit 片上 RAM 与 160 个 DSP。本设计最紧缺的资源是片上 RAM：一帧 640×480 的 32 位图像约 4.9 Mbit，超出全部片上容量，因此帧缓冲、素材与命令表都必须驻留 DDR3（256 MB，16 位颗粒 @ 800 Mbps，控制器 AXI 数据宽 128 位、地址 28 位）。芯片上的 RAM 预算优先投给两处跨时钟 FIFO 与引擎的读写缓冲，DSP 预算集中投给像素混合通路。

DDR3 地址空间按“谁写、谁读、写多少次”划分，如图 2 所示。程序区约 4 MB；两张各 1 MB 的命令表支持双表流水；素材区 16 MB 存放启动时从 Flash 装入的纹理图集与字库，CPU 只写一次、引擎只读；两个 4 MB 帧缓冲构成双缓冲对；另设 8 MB 离屏层供背景预合成与调试抓帧。Flash 8 MB 恰好放满比特流（2 MB）、应用（512 KB）与素材包（4 MB），上电由 bootloader 引导应用、应用再经 SPI0 把素材装入 DDR3。

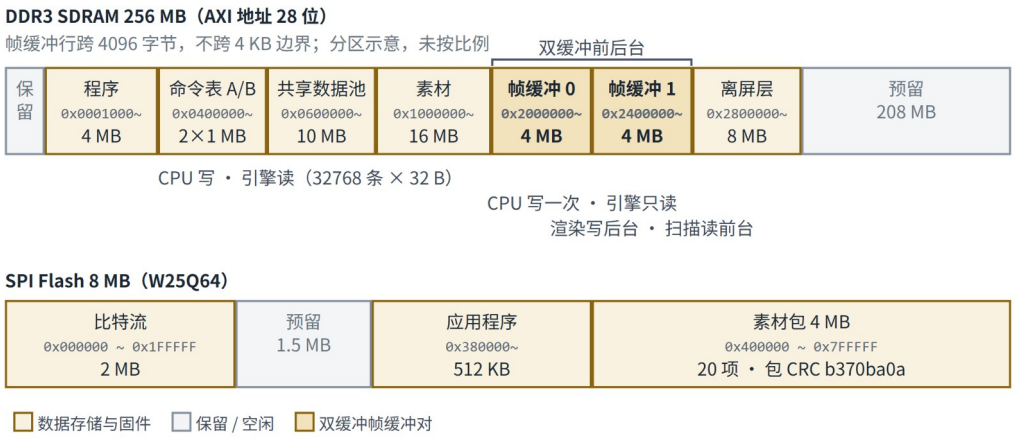


图 2　DDR3 与 SPI Flash 的地址分区

像素格式选 32 位 XRGB8888（帧缓冲）与预乘 ARGB8888（纹理），一个 128 位 AXI 拍正好装 4 个像素，搬运与混合都按 4 像素对齐。帧缓冲行跨取 4096 字节，一行 640 像素恰好一次突发、永不跨 4 KB 边界——AXI4 协议禁止突发跨越 4 KB 边界，这个约束在数据布局层面被直接消掉，硬件不再需要任何边界检查逻辑；地址计算也退化为移位。这两条设计决策都源自开工前对控制器与协议的定量实测，是本设计“预算先行”方法的第一处落地。

2.2.2 总线接入与存储仲裁

引擎作为独立主机直接访问内存，经 Axi_Mux 三端口接入。接入前用探针主机实测了 DDR3 控制器的事务模型：突发上限 256 拍，每笔事务周期数 $T(N) = t_tx + k \cdot N$，其中写事务固定开销 $t_tx = 7.25$ 周期、每拍增量 $k = 1.183$；读事务固定开销高达 27.41 周期、增量 1.086。也就是说，一笔只读 1 拍的事务要花 29 个周期，摊到每拍的开销是 256 拍满突发情形的约 24 倍。持续吞吐实测为每周期 0.85~0.92 拍，折合带宽远超扫描输出与整帧读写的合计需求——带宽不是瓶颈，事务次数才是。这条实测结论决定了引擎的一切访存形态：读方向避免小块读，源与目的按行合并成长突发；写方向按行凑满 160 拍。

多路器的仲裁行为也以实测为准。直接仿真厂商多路器的授权状态机发现：三路持续请求时 S0、S1 交替而 S2 陷入饥饿（starvation），厂商所称“优

先级模式”并非严格优先级。本设计不依赖对厂商逻辑的猜测，而是用三层措施形成边界：S0 扫描与 S1 引擎各自的一笔在途契约（由引擎 RTL 保证）、永久模块 axi_mux_guard 对 S2 地址等待的老化监测（超过 64 个周期即从新授权中暂时屏蔽 S1）、以及显示 FIFO 的水位与欠载计数。板上三方并发实测：扫描 FIFO 在探针满负载读、满负载写与空闲三种负载下最低水位分别为 347 / 331 / 327 拍，触底与欠载均为零；整机接入引擎后按同一判据复测最低 335 拍，压力搜索全程最低 332 拍、欠载为零。

CPU 侧的数据缓存不含一致性机制，引擎直接写 DDR3 会与缓存副本脱节。本设计把共享区使用规则定为“读回前必须失效”，并通过机理实验标定了代价：失效指令本身全清约 67 周期、单地址约 4 周期，失效后每行首次访问重新装行约 69 周期。整帧读回约 13 ms（估算），因此逐帧 CRC 对比作为调试与验收手段按需开启，不进入生产帧路径；游戏运行期需要引擎结果的场合一律走寄存器与中断。

2.2.3 扫描输出与双缓冲

显示链路如图 3 所示。display_fetch 按异步 FIFO 水位发起 160 拍的行读（一行 640 像素恰好 160 拍），数据经 128 位×512 深的异步 FIFO 从 100 MHz 总线域跨到 25.2 MHz 像素域；display_out 把每个拍拆成 4 个像素、处理对齐相位，并按 SRC 寄存器选择输出彩条或帧缓冲；hdmi_tx 完成 TMDS 编码。上电默认输出八色彩条，显示基线不依赖任何软件——这也是一条独立的回归判据：无论渲染侧如何改动，彩条必须始终正确。

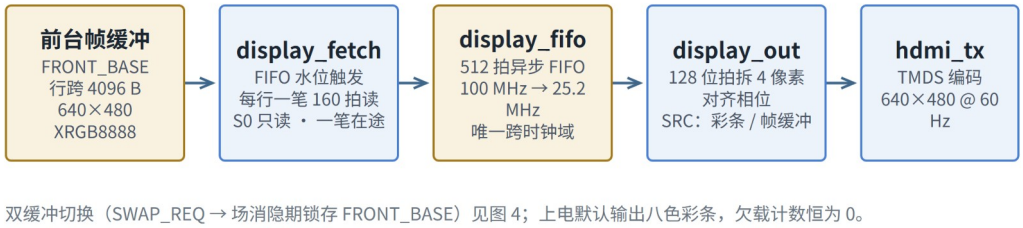


图 3　从 DDR3 帧缓冲到 HDMI 的扫描输出通路

字节序是一个容易被忽视的正确性风险：小端内存中 128 位拍内 4 个像素的字节排列与编码器输入的位段必须严格同序。本设计先经推导确认“拆像素不需要任何字节交换”，再在板上用四条横向色带实验做了端到端验证，屏幕自上而下呈现红、绿、蓝、白与推导一致。

双缓冲与场同步的时序如图 4 所示。软件写 BACK_BASE 与 SWAP_REQ 后，切换在下一个场消隐期由硬件锁存完成，请求位自清；若引擎未能在消隐前渲染完，该场保持前缓冲并被计为一次掉帧——帧率统计因此有客观定义（场数减切换数），不依赖目测。两份命令表与双缓冲配合形成两级流水：CPU 填表 A 时引擎渲表 B，消隐时呈现 B 的结果，画面永远完整。

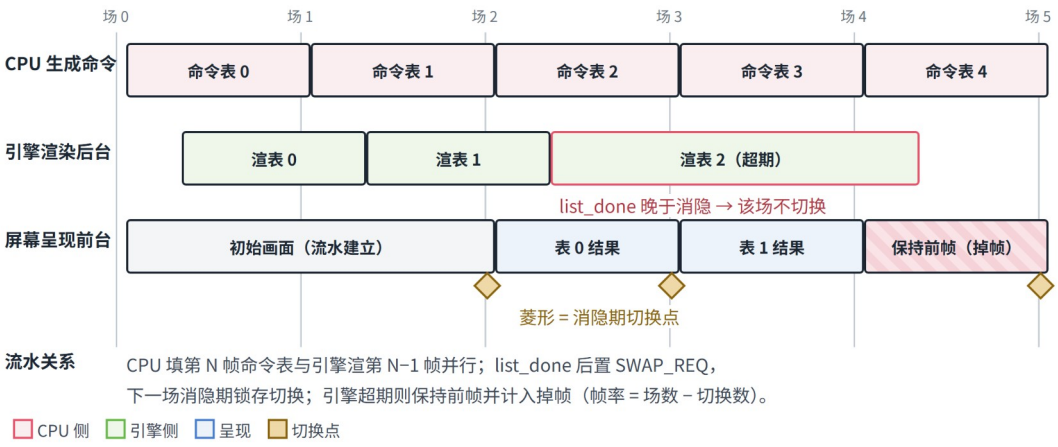


图 4　双缓冲与两级命令表流水的场同步时序

2.2.4 2D 渲染引擎

引擎是本作品的核心硬件，内部结构如图 5 所示，自上而下五个部分：APB 从机与寄存器组、命令取数、检查与解码、几何层、像素流水线，底部是 AXI4 事务层。所有“地址”经表面表间接寻址：驱动先在 SURF[0..15] 登记表面的基址、宽高、行跨、地址空间与像素格式，命令只携带表面号与坐标，硬件统一完成裁剪与越界检查。这既满足赛题“指令包含源地址、目的地址、宽度、高度”的要求（源/目的地址由表面基址与坐标换算给出），又让非法引用在进入像素通路之前就被拦截。

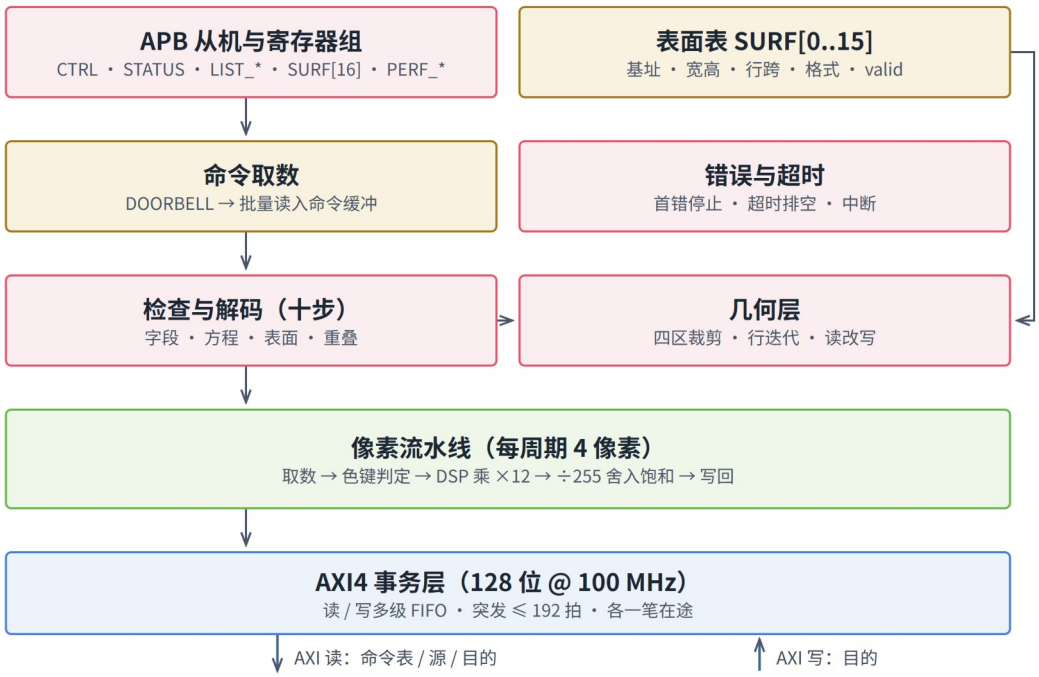


图 5　渲染引擎的分层结构

命令由 DOORBELL 寄存器受理后按 LIST_BASE 从 DDR 批量读入片上缓冲，LIST_COUNT 为硬边界，END 可提前结束。命令记录固定 32 字节 8 个字，记录长度取 2 的幂使硬件用移位即可定位第 k 条命令。操作码共五种：NOP、FILL（纯色填充）、BLIT（图像块拷贝）、SET_CLIP（设置持续生效的裁剪框）与 END（提前结束符）；w0 内嵌混合方程编号、色键使能与常数 alpha，w1~w5 携带源/目的坐标、宽高、常量源像素与色键颜色。命令提交后由固定十步检查（操作码、字段、方程、源表面、目的表面、地址空间、格式、预乘约束、重叠）把关，首错停止：失败命令不产生任何像素副作用，错误码与出错记录索引锁存在寄存器中并触发中断，软件可精确定位到表内第几条命令的哪个字段。总线类错误（读/写响应非 OKAY、事务超时）同样进入该通路，超时后引擎排空在途事务再回到空闲，保证 DDR3 上同时运行的处理器不受牵连。

像素流水线如图 6 所示，每级处理一个 128 位拍（4 像素）：取数、色键判定、DSP 乘、加与舍入、写回。混合统一为 out = src×Fs + dst×Fd，六种方程

共用同一条通路：COPY（拷贝）、ALPHA（预乘半透明，主路径）、ALPHA_STRAIGHT（非预乘对照路径，直接实现赛题公式）、ADD（加法发光）、CONST（整屏常系数）与 MUL（乘法暗化）。主路径纹理预乘 alpha 后每通道只剩一次乘法；12 个混合核（4 像素 × RGB 三通道）各占用一块 DSP，与赛题“利用内部 DSP/硬件乘法器”的要求逐字对应。除以 255 的舍入采用快速式 out = min(255, (t + (t>>8)) >> 8)，其在全定义域与 half-up 精确商逐值一致（穷举校验），中间量保留 17 位避免模截断误差。色键按源表面的 32 位存储字整字比较，命中像素不写回，实现不规则边缘精灵的完美叠加；混合只作用于 RGB 三通道，写回 XRGB8888 帧缓冲时高字节强制为 0。

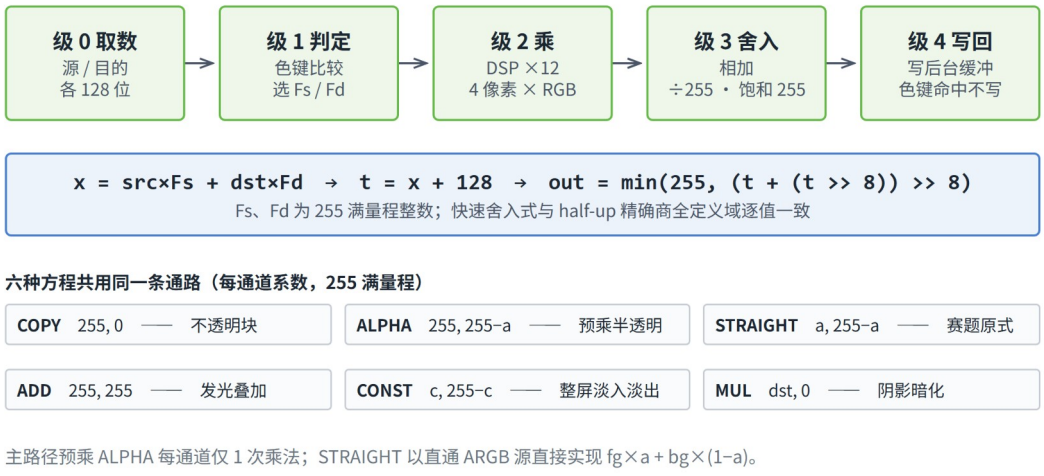


图 6　像素流水线与混合方程

引擎内建九个性能计数器（命令数、像素数、色键跳过数、读写突发数、读写 beat 数、等 DDR 周期数、忙周期数），并保证可对账约束（如读 beat 数等于各读突发长度之和）——计数器不是装饰，而是第 2.4 节验收判据的数据来源。

2.3 软件系统介绍

2.3.1 软件整体：分层与双目标构建

软件侧的分层如图 7 所示：应用层（游戏与判据程序）、算法层（lib/，纯 C）、板级支持层（bsp/）与厂商层（sdk/，SoC IP 生成物，只读）。分层的可见范围由编译系统的头文件搜索路径强制：算法层看不见任何平台头文件，应用层看不见 sdk，越界引用直接编译失败，并有脚本以故障注入方式验证这条性质本身。

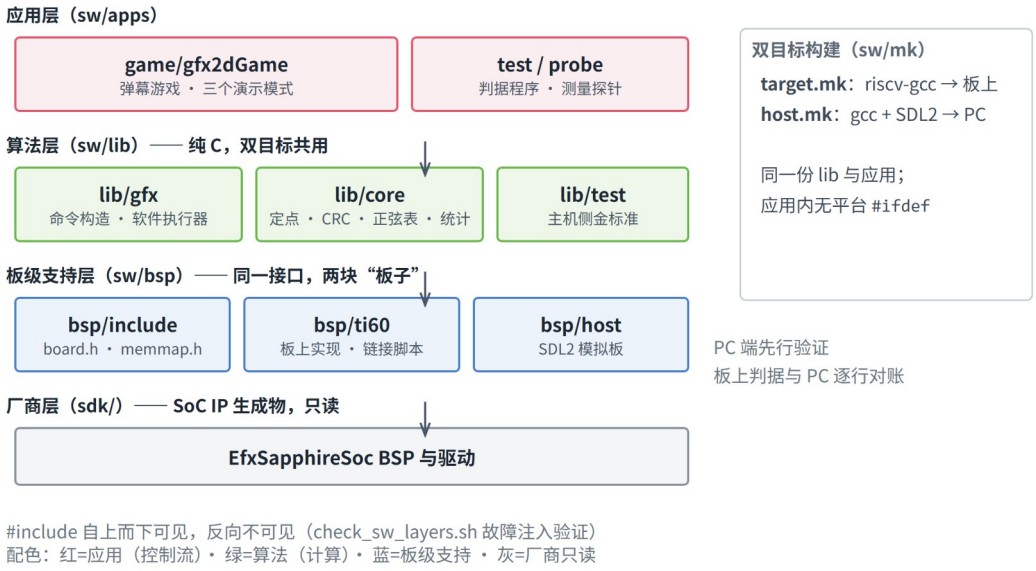


图 7　软件分层与双目标构建

这套结构带来一个关键能力：同一份算法层与应用层代码可以直接编译为 PC 程序。bsp/host 用 SDL2 提供窗口、键盘与“内存模拟的 DDR 分区”，PC 版游戏先于板上开发与调试；板级接口 board.h 在两端语义一致（场节拍、呈现、输入、串口），使“PC 上逐行对账、板上复现”成为常规操作。游戏开发、命令表构造、统计埋点都在 PC 上完成，板子只承担最终验证——这把一块开发板变成了一整个开发平台。

软硬件的分工在帧循环里一目了然：处理器负责游戏逻辑（实体池更新、弹型参数化发射、碰撞检测、关卡与规则）、命令表构造与引擎调度；引擎负

责一切逐像素操作。处理器侧不做任何逐像素运算，这是“把 CPU 从像素搬运中解放出来”的赛题命题在本设计中的直接回答。

2.3.2 软件各模块

命令表构造层与软件执行器。 构造层把游戏的绘制意图翻译成命令记录：表面登记、矩形提交、裁剪管理、命令表滚动缓冲。软件执行器按与硬件完全相同的十步检查和逐像素规格解释同一份命令表，它有三重身份：PC 上的渲染后端（游戏可玩）、硬件的参照实现（金标准）、以及统计埋点宿主（逐帧像素数、按方程分布）。板上模式支持在软件后端与硬件后端之间一键切换，HUD 同屏显示两路帧率，这正是赛题要求的“实时对比”。

素材流水线。 如图 8 所示，PNG 图集与开源字库（Spleen，BSD-2 许可）经打包脚本解码为预乘 ARGB 图集并附目录头，素材包 2,840,320 字节、包校验码 b370ba0a；与比特流、应用镜像合成为 Flash 镜像一次烧写。上电后 bootloader 从 0x380000 引导应用，应用经 SPI0 把素材装入 DDR3 素材区并逐项校验，最后把表面登记进引擎。装载通路对坏包显式拒绝（九项判据），素材包在新字库引入后重打包三次板上回归全过。

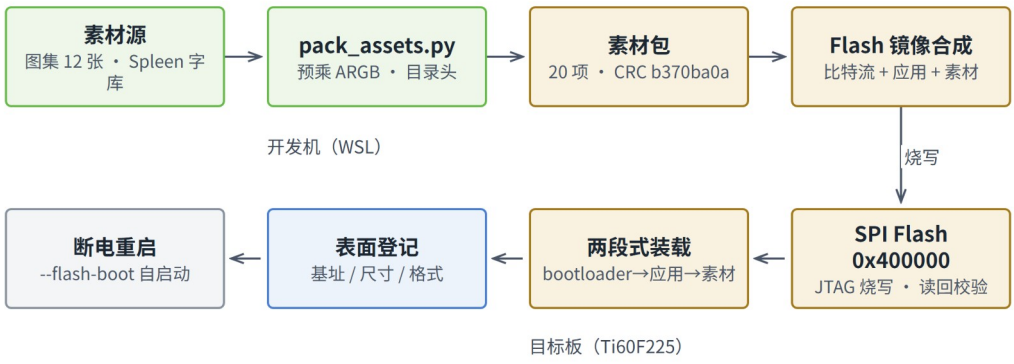


图 8　素材流水线与启动装载

游戏本体 MYRIAD。 竖版弹幕射击，左侧 400×480 游戏区、右侧 240×480 HUD（分数、残机、活动弹数、软/硬件帧率）。数值全程定点：位置与速度 Q.8 定点，角度用 16 位二进制角查 1280 项正弦表，避免无 FPU 处理

器上的浮点开销。实体采用四池管理（敌弹、自弹、粒子、拾取物）与离屏立即回收，更新循环无函数指针调用、无动态分配。弹型由发射器参数方程生成（圆环、螺旋、自机狙、扇形、随机雨），关卡为“30 秒杂兵 + 四张符卡”的 Boss 战结构，终符的密度随时间上升，兼作压力测试负载。碰撞为 O(N) 圆形判定，自动炸弹与残机规则完整，输入支持板上按键与上位机经串口转发的键盘/手柄。

三个演示模式。 模式 1 基础演示：大量方块随机移动与多张大图快速切换两个场景，屏幕同屏显示软件与硬件两路帧率；模式 2 游戏：完整可玩关卡，默认 Alpha + 色键；模式 3 压力测试：以终符为负载，自动执行“N_max 搜索”，把结果（含搜索轨迹）实时显示并记录。三模式一键切换，满足从评审演示到量化验收的全部场景。

2.4 验证体系

本设计的验证围绕一个中心思想展开：软件执行器与硬件流水线是同一份规格的两个实现，规格本身用穷举与独立实现钉死，两个实现之间用逐像素对比闭环。如图 9 所示，验证分四层。

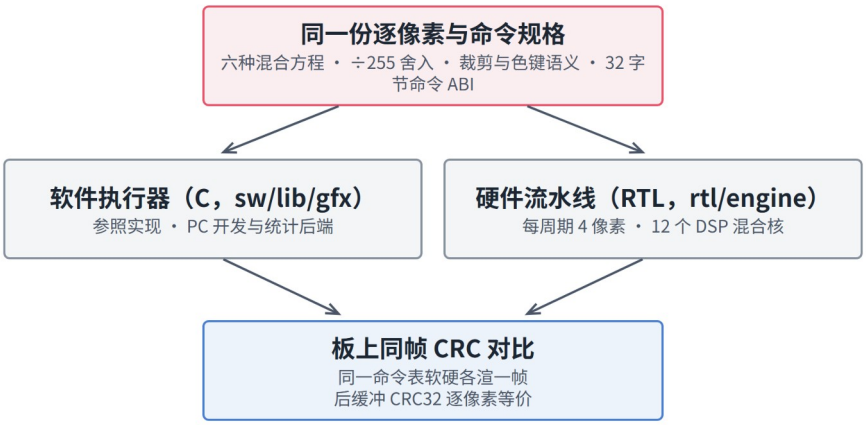


图 9　以同帧 CRC 为核心的四层验证体系

规格层：逐像素规格把“混成一个通道的数值”写到没有第二种读法——六种方程的系数、÷255 的舍入顺序、中间量位宽、裁剪的四区交集、色键的比

较域、字节序全部定稿，其中舍入与位宽由全定义域穷举校验支撑（ALPHA 预乘路径 8,421,376 组逐值一致；快速舍入式与精确商全域等价）。命令语义另以 159 条带字面期望值的一致性向量校验，覆盖字段矩阵、检查优先级、错误码与失败副作用，向量工具自带六项故障注入自检。

单元层：引擎各模块在仿真中验证，AXI 从机模型带随机反压与可配延迟（测试台不得比真实从机更强）；每个判据都配有故障注入对照用例，注入故障后判据必须报错；语义类判据进一步以变异测试（Mutation Testing）方式系统性构造故障样本，未被检出的变异即为判据缺口。

系统层：十一帧金标准用例覆盖六种方程、色键、裁剪与越界，板上同一命令表软硬各渲一帧、比较后缓冲 CRC32，全帧逐像素等价；性能版在真实游戏命令表上十组 CRC 全部相同，而故意跳过缓存失效的故障注入版十组全部不同——证明对比本身有效。

性能层：硬件 PERF 计数器与软件统计按同一统计定义采集，约束关系逐项对账；扫描 FIFO 水位、欠载计数与中断故障计数构成实时性的客观证据。全部性能数字以场数减切换数的帧率定义记录，与 HUD 显示一致。

第三部分　完成情况及性能参数

3.1 整体介绍

系统实物为一块 Ti60F225 开发板通过 HDMI 连接显示器，上位机经板载 USB 串口承担日志与输入转发；上电即从 Flash 自启动进入游戏，无需任何加载操作。下两图为开发板与显示器的实拍（厂商资料图），以及系统实际运行的屏摄：显示器上 MYRIAD 正以硬件后端运行，HUD 显示 HW FPS 61，桌面下方为开发板与上位机连线。

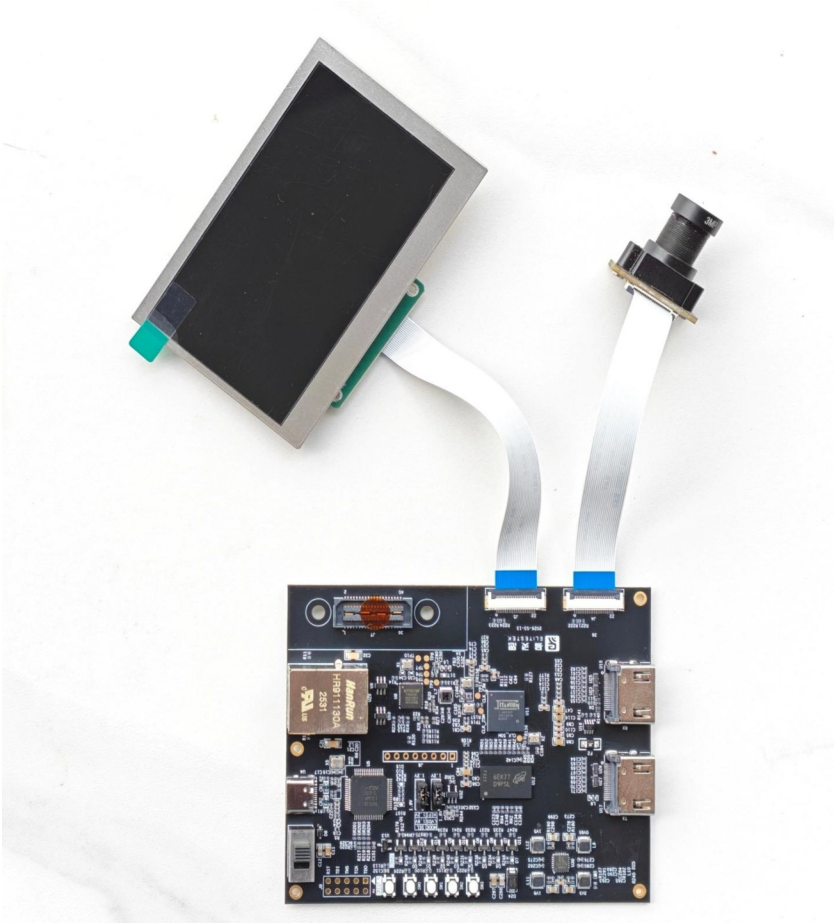


图 10　Ti60F225 开发板与显示器实拍（图片来源：易灵思《Ti60F225 DEMO 板软硬件设计说明》第 2 页）



图 11　系统运行屏摄：开发板经 HDMI 驱动显示器运行 MYRIAD，硬件后端 HUD 显示 61 FPS

3.2 工程成果

3.2.1 硬件成果

整机设计在 Efinity 2026.1 下编译，映射、布局布线、时序与比特流生成四步全部通过，无约束告警、无时序违例。表 2 给出整机资源占用及与平台基线的对比：自研部分在 60K 逻辑单元的芯片上以不到四成的逻辑增量实现了完整渲染引擎，DSP 仅占用 10%，为画面效果扩展留出余量。

表 2　整机资源占用与平台基线对比（实测）

资源	平台基线（SoC+总线+显示）	整机（含引擎）	占芯片总量
XLR 逻辑单元	16151	36009	59.2%
片上 RAM（10 Kbit/块）	65	105	41.0%
DSP	4	16	10.0%
core_clk 时序余量（setup）	+3.405 ns	+1.067 ns	无违例

整机中引擎新增的 12 个 DSP 与像素流水线的 12 个混合核一一对应；新增 RAM 主要是扫描输出异步 FIFO 与引擎读写缓冲，均与预算一致。

3.2.2 软件成果

软件侧交付三部分：可玩的弹幕游戏 MYRIAD（板上 60 FPS 运行）、PC/板上双目标软件平台（同一份代码在两端逐行对账运行）、以及覆盖从素材打包到 Flash 自启动的完整交付链（应用区写入、独立读回校验、断电重启自启动均已在板上验证）。三个演示模式的画面如下——按双目标的分工，游戏逻辑与 HUD 统计以 PC 解释器运行画面展示（与板上运行同一份代码、逐行对账一致）；板上硬件后端的实际运行以 3.1 节屏摄与 3.3 节实测数据证明。

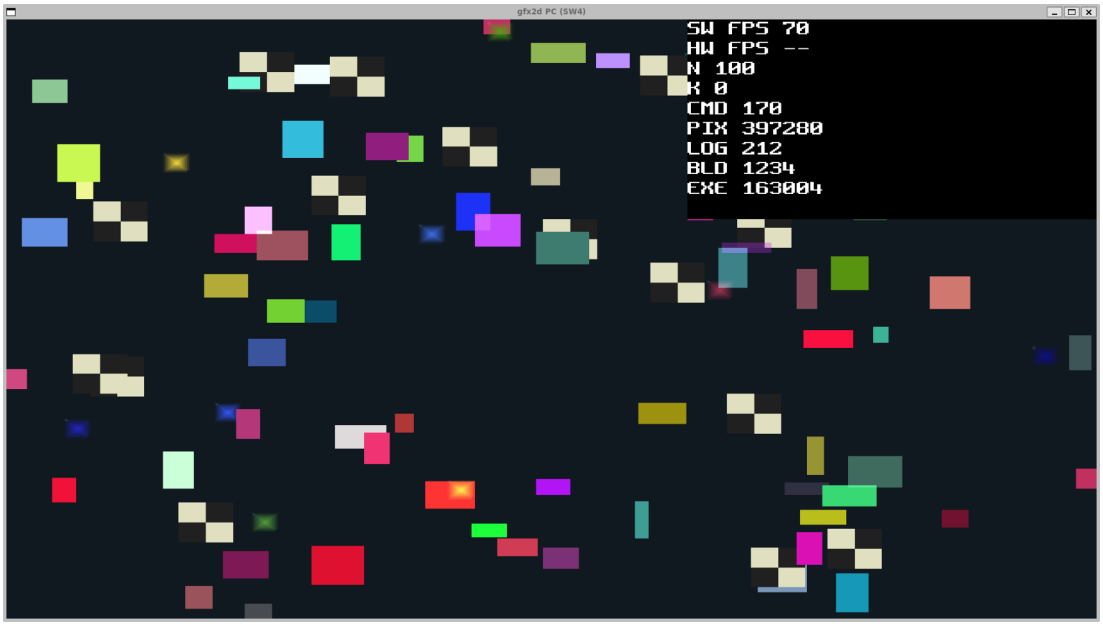


图 12　模式 1 基础演示：方块随机移动场景，HUD 同屏显示软件与硬件两路帧率（PC 解释器运行截图）

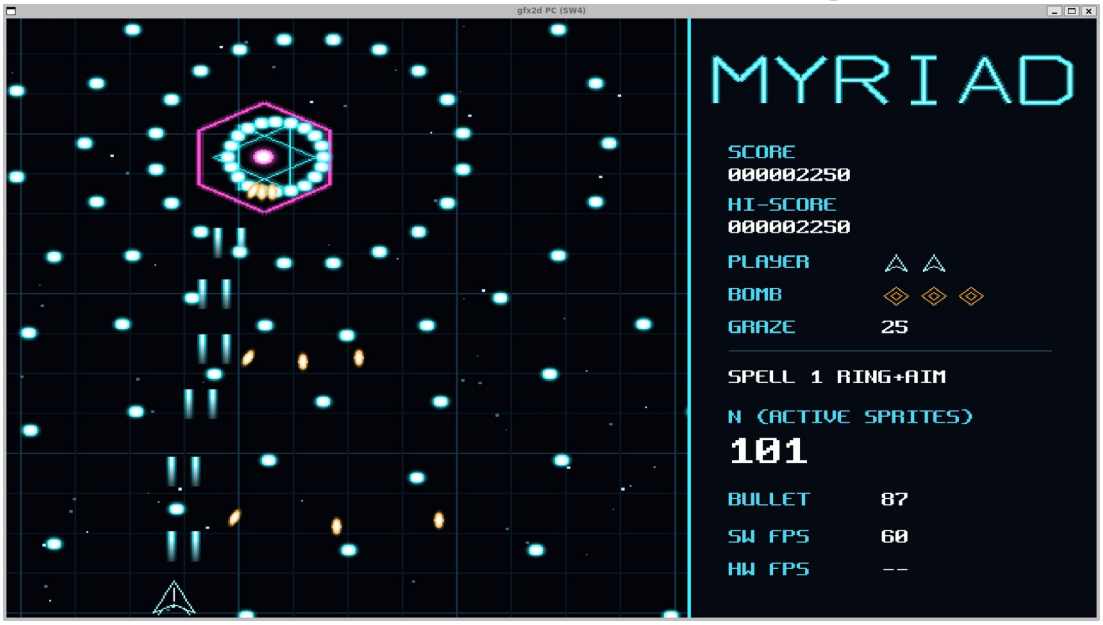


图 13　模式 2 弹幕关卡：Boss 符卡战画面，敌弹为预乘 Alpha 半透明精灵，色键去除背景（PC 解释器运行截图）

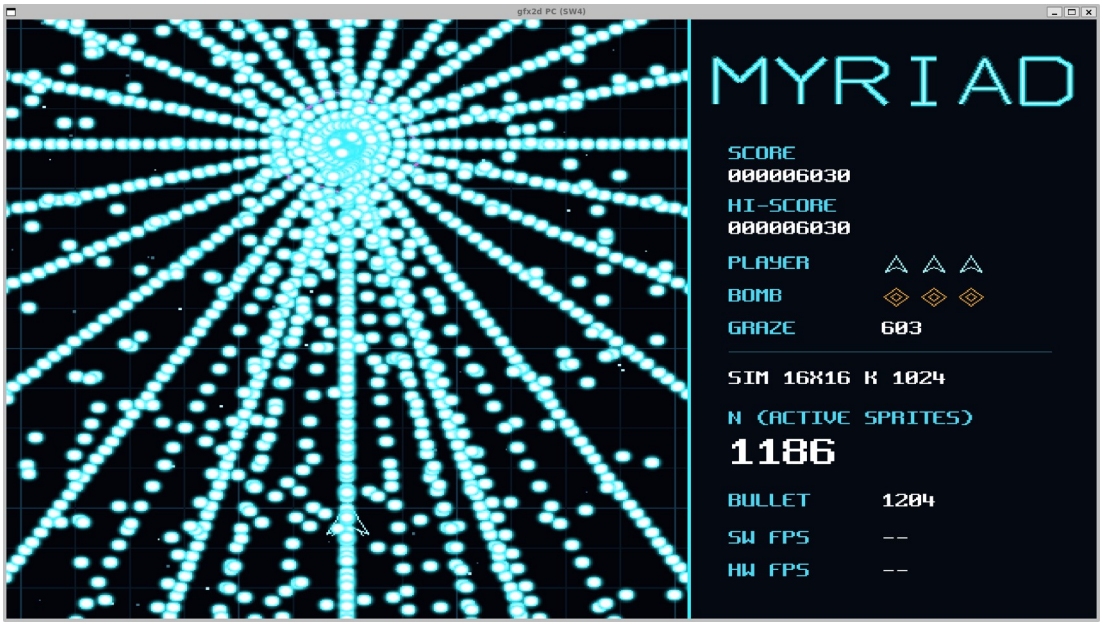


图 14　模式 3 压力测试：K1024 压力档下 1186 颗活动弹（PC 解释器运行截图，模拟时钟）

3.3 特性成果（量化指标）

3.3.1 软硬件渲染帧率对比

图 15 给出同一游戏、同一命令表在两个后端下的实测帧率。普通游戏画面：硬件后端首秒预热后稳定 60 FPS；纯软件后端约 2.5 FPS（单帧最长 38.45 M 周期，即约 384 ms），加速约 24 倍。压力负载 S1（负载旋钮 1744，约 2030 颗活动弹，约为稳态极限档两倍的规模）：硬件后端 60 秒渲出 1839 帧、约 30.6 FPS，期间 FIFO 零欠载；同规模软件后端大量掉帧，场上存量仅约 650 颗。该对比直接回应赛题“在屏幕上实时对比显示纯软件渲染与硬件加速渲染的帧率差异”的要求：演示时屏幕 HUD 即同屏显示这两个数字。板上硬件后端的实际运行见图 16：屏摄中 HUD 显示 HW FPS 61、活动弹 292。

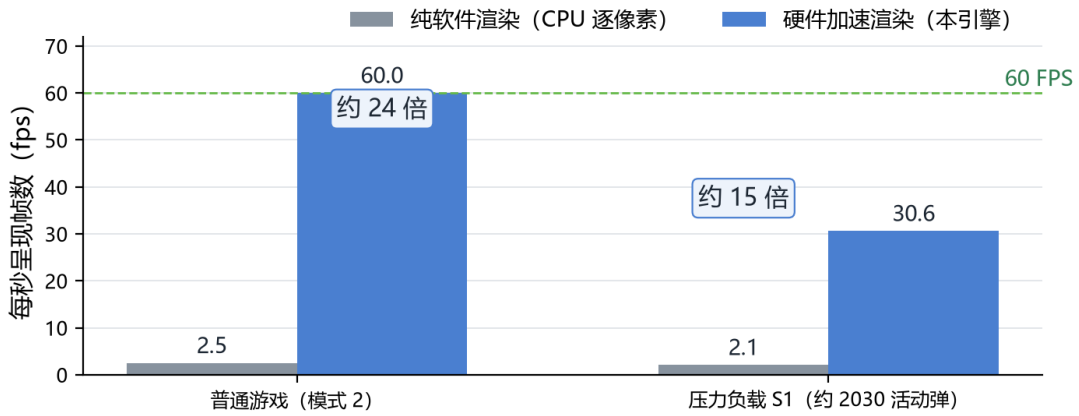


图 15　同一负载下纯软件渲染与硬件加速渲染的实测帧率

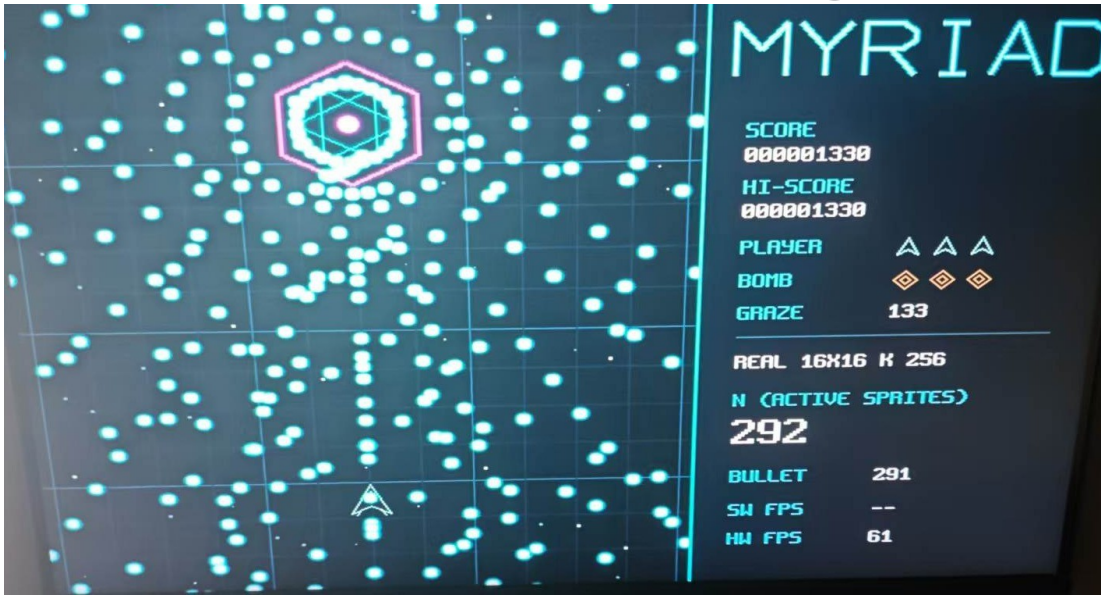


图 16　板上硬件后端运行屏摄：HUD 显示 HW FPS 61、活动弹 292（REAL 16X16 K 256 档）

3.3.2 同屏活动精灵数量极限（N_max）

赛题高阶挑战要求“在开启硬件加速（Alpha 混合 + Color Keying）的情况下，挑战保持 60 FPS 稳定运行时的同屏活动元素数量极限”。本作品把该指标定义为可自动执行的判据：以真实游戏终符为负载，负载旋钮 K 控制每 125 帧发射的弹数；搜索采用倍增到首次失败、再二分收敛的策略，每档短测 120 场，候选档再以 3600 场（60 秒）确认，掉帧判据为帧时间超过 16.67 ms 预算。搜索全程扫描 FIFO 最低水位 332 拍、欠载为零。

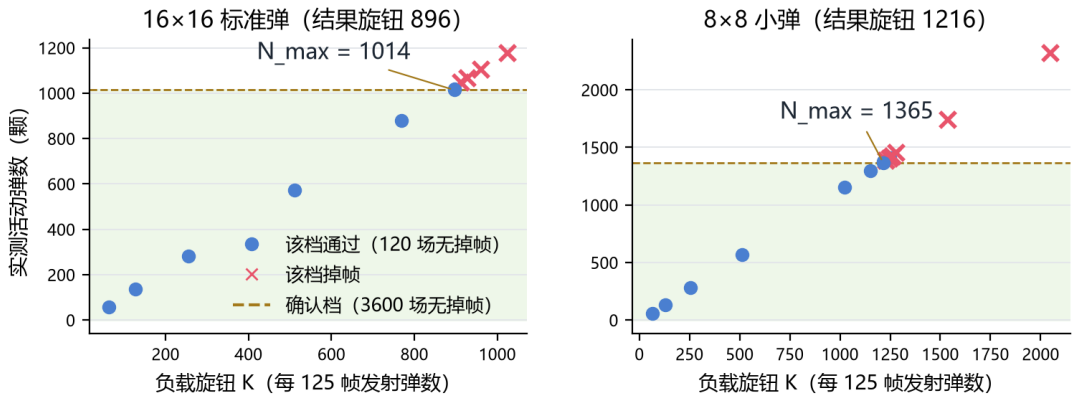


图 17　真实时钟下 N_max 的自动搜索轨迹（左：16×16；右：8×8）

实测结果：16×16 标准弹 N_max = 1014 颗（旋钮 896，确认档 3600/3600 场零掉帧，最长帧 1.638 M 周期）；8×8 小弹 N_max = 1365 颗（旋钮 1216，确认档 3600/3600 零掉帧，最长帧 1.191 M 周期）。作为对照，同一负载下纯软件后端连初始 64 颗档都无法维持一场一帧（记 BELOW_N0）——硬件引擎把可玩弹幕密度提高了一个数量级以上。图 18 展示固定档 K896（约 1014 颗活动弹）连续 60 秒的逐秒帧时间与活动弹数：弹幕约 4 秒爬坡进入稳态后，帧时间稳定在 1.63 ms 附近，距 16.67 ms 预算有约 10 倍余量，无累积劣化；60 秒内仅首秒因爬坡记 59/60，其余 59 个窗口全部 60/60。

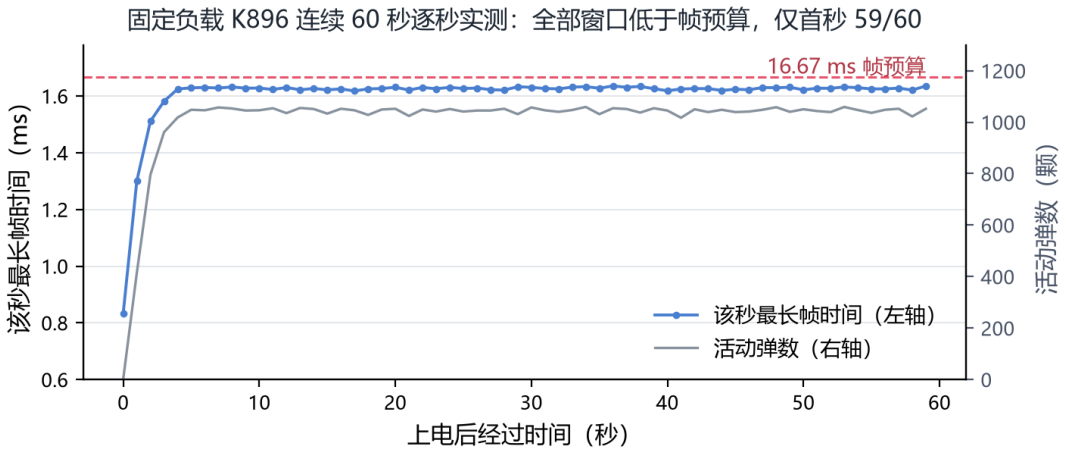


图 18　固定负载 K896 连续 60 秒的帧时间与活动弹数曲线

3.3.3 赛题要求逐条对照

表 3　赛题要求逐条对照

赛题要求	本作品实现	实测证据
解析渲染指令（操作类型、源/目的地址、宽、高）	32 字节命令，五种操作码，表面表间接寻址	159 条语义向量全过；板上十一帧金标准 CRC 一致
图像块拷贝与纯色填充	BLIT / FILL，硬件裁剪与越界检查	同帧 CRC 逐像素等价
HDMI 输出不低于 640×480@60	TMDS 640×480@60，上电彩条基线	FRAME_COUNT 每秒 +60；displayTest 59.9 FPS
总线直接访问内存，突发与 FIFO	AXI4 128 位主机，突发 ≤192 拍参数化，多级 FIFO	突发上限 256 拍实测；FIFO 欠载恒为 0

赛题要求	本作品实现	实测证据
C 驱动配置寄存器	APB 寄存器组 + 命令表 + DOORBELL + 中断	板上驱动全链路运行
双缓冲防撕裂	场消隐硬件锁存切换，掉帧有客观定义	SWAP_COUNT 与场数对账；切换仅发生于场消隐期
屏幕实时对比软/硬帧率	HUD 同屏显示两路帧率，一键切换后端	60 FPS 对 2.5 FPS（约 24 倍）
硬件 Alpha 混合（DSP）	12 个 DSP 混合核，预乘主路径 + 赛题原式对照路径	ALPHA 预乘 8,421,376 组穷举一致；整机 16 DSP
Color Keying	按源格式整字色键，命中跳写	KEY_SKIP 计数器对账；色键精灵板上渲染正确
高负载游戏 Demo，60 FPS 下冲同屏精灵极限	真实弹幕游戏 + 自动 N_max 搜索	16×16：1014 颗；8×8：1365 颗；3600 场零掉帧

3.3.4 可靠性证据

全部性能日志在板上采集并入库：压力搜索全程中断故障计数为 0、扫描欠载为 0；通关录像 9208 帧在板上以硬件后端串行回放，与同版本 PC 参考回放逐行一致；应用 Flash 区写入后经独立读回校验，断电重启自启动验证通过。上述证据共同支撑一个结论：指标是在真实游戏负载、真实时钟下长时间稳定复现的，并非单次演示的最佳成绩。

第四部分　总结

4.1 可扩展之处

命令 ABI 在设计时即预留了扩展空间：混合方程表可新增行而不改流水线结构；着色（tint）字段已定义，启用后每通路增加一组乘法；INDEX8 索引色与调色板寄存器页已保留。帧缓冲行跨为参数化设计，提高到 720p 只需更换像素时钟并放宽行跨。片上缓存方面，16 KB/4 路的数据缓存候选已完成同程序对照实测（每弹幕更新开销 3.51 ms 降至 2.34 ms/2000 颗），可作为后续构型选项。架构上预留的条带缓冲与纹理缓存地址空间，可支撑“按条带合成”的第二执行模式，将每帧 DDR3 固定事务数与精灵数量解耦。

4.2 心得体会

这个项目给我们最大的收获是“先定量预算、后架构设计”的工作方式。开工前我们用探针程序实测了 DDR3 控制器的事务模型，发现读事务的固定开销接近写的四倍——正是这条实测数据，而不是直觉，决定了帧缓冲行跨取 4096 字节、决定引擎读方向只走长突发。后来整机联调时，像素通路几乎不需要为访存效率返工，预算的准确性回报了前期投入。

第二点体会是验证体系的杠杆作用。赛题要求软件与硬件渲染“结果一致、帧率可比”，我们把它翻译成工程语言：同一份逐像素规格、两个实现、逐像素对比。规格定稿阶段为“除以 255 怎么舍入”这类细节做过全定义域穷举，当时看似苛刻，但正是这份规格让 PC 上的软件开发与板上硬件并行推进、最终一次对账即通过，省掉的返工远多于投入。

第三点体会是“让数字可对账”。帧率不是目测的，是场数减切换数算出来的；FIFO 是否实时，看最低水位与欠载计数；引擎忙不忙，看 PERF 计数器。评审问到的每一个数字都能指到一条日志，这种踏实感是本作品最想传达的品质。

最后，把一个真实可玩的游戏当作验收负载，让所有优化都有了共同的标尺。弹幕游戏对吞吐的渴求没有止境，它逼着我们把每一个周期都花在刀刃上；而游戏的趣味性又让技术成果在演示时一目了然——工程与作品感，在这道赛题上并不矛盾。

第五部分　参考文献

[1] 全国大学生嵌入式芯片与系统设计竞赛组委会. 2026 FPGA 创新设计赛道选题指南（易灵思）[Z]. 2026.

[2] Efinix Inc. Titanium Ti60F225 FPGA Datasheet v2.0 [Z]. 2024.

[3] Efinix Inc. Sapphire RISC-V SoC User Guide v8.2 [Z]. 2025.

[4] Efinix Inc. DDR3 Soft Controller Core User Guide v2.1 [Z]. 2024.

[5] Efinix Inc. Efinity Software User Guide (2026.1)[Z]. 2025.

[6] ARM Ltd. AMBA AXI and ACE Protocol Specification, IHI 0022E [S]. 2010.

[7] RISC-V Foundation. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA [S]. 2019.

[8] T. Porter, T. Duff. Compositing Digital Images [C]// SIGGRAPH '80: Proceedings of the 7th Annual Conference on Computer Graphics and Interactive Techniques. 1980: 253–259.

[9] J. L. Bentley, A. C.-C. Yao. An Almost Optimal Algorithm for Unbounded Searching [J]. Information Processing Letters, 1976, 5(3): 82–87.

[10] STMicroelectronics. AN4943: How to Use the Chrom-ART Accelerator (DMA2D) on STM32 MCUs [Z]. 2018.

AI 工具使用说明

本作品在资料检索与整理、文档文字校对与排版、插图配色与样式调整等辅助性环节使用了 AI 工具。系统方案设计、全部硬件 RTL 与软件代码的实现、实验设计以及所有上板实测数据，均由团队成员独立完成。